Pilhas e Filas

Algoritmos e Estruturas de Dados

Prof. Dr. Carlos Melo

carlos.melo@upe.br

UPE UNIVERSIDADE DE PERNAMBUCO
CAMPUS GARANHUNS

Pilhas

- Quando pensamos no conceito de **pilhas** logo nos vem a cabeça um pack de coisinhas disponíveis no nosso dia a dia.

- Do inglês a definição *stack* exclui as pilhas do controle remoto da nossa análise.

- Mas as pilhas de roupas e de pratos são a base que precisamos para criar essa estrutura de dados universalmente utilizada.

Pilhas

- O TAD da pilha nos delimita o caminho para construir uma estrutura de dados em que eu opero somente no topo.

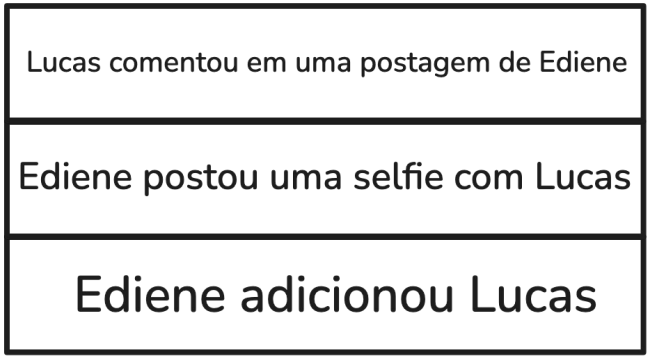
- Se quero algo da base, preciso mover elemento por elemento até alcançá-lo.

- Se quero adicionar um novo elemento sempre o faço no topo da pilha.

Pilhas

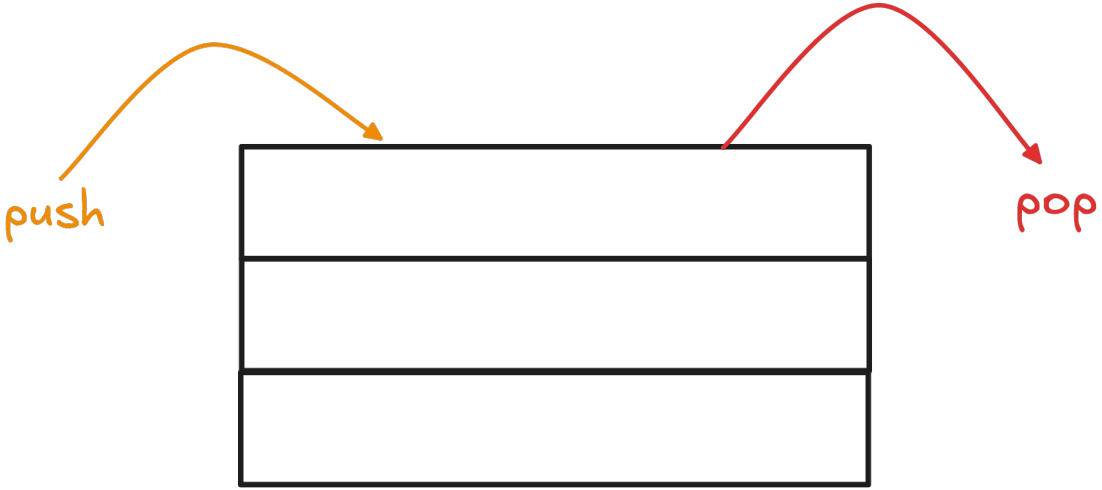
- Pilhas são ideais quando nossa preocupação é unicamente com o elemento mais recente adicionado, ou quando a ordem que vemos e salvamos elementos importa.

- Em uma rede social o que está no topo é acessado mais frequentemente.



Pilhas

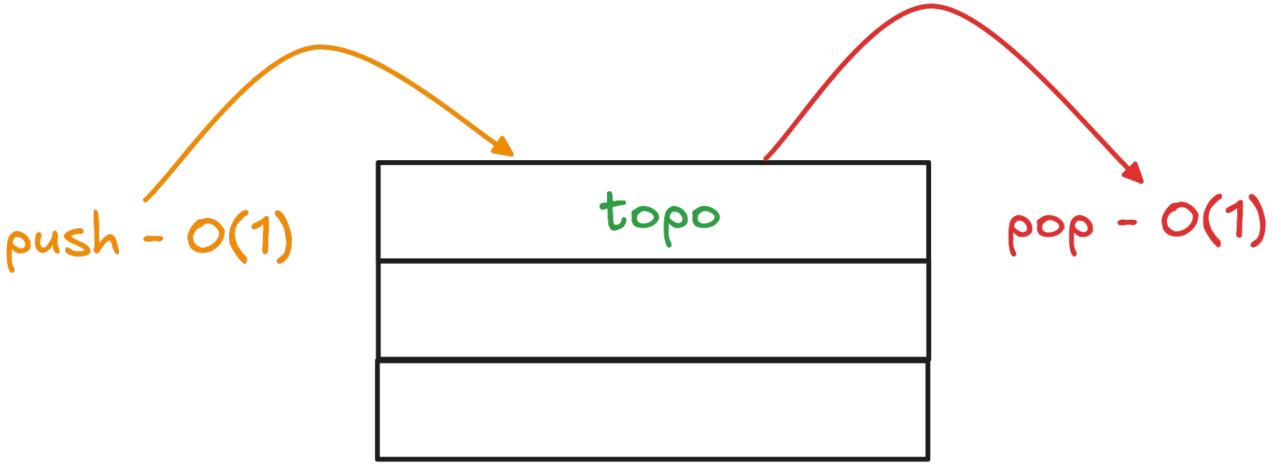
- Quando adicionamos um elemento em uma pilha realizamos a operação de Push (empurrar) ao invés de Insert (inserir)
- Quando removemos um elemento da pilha a operação é o pop e não remove



carlos.melo@upe.br

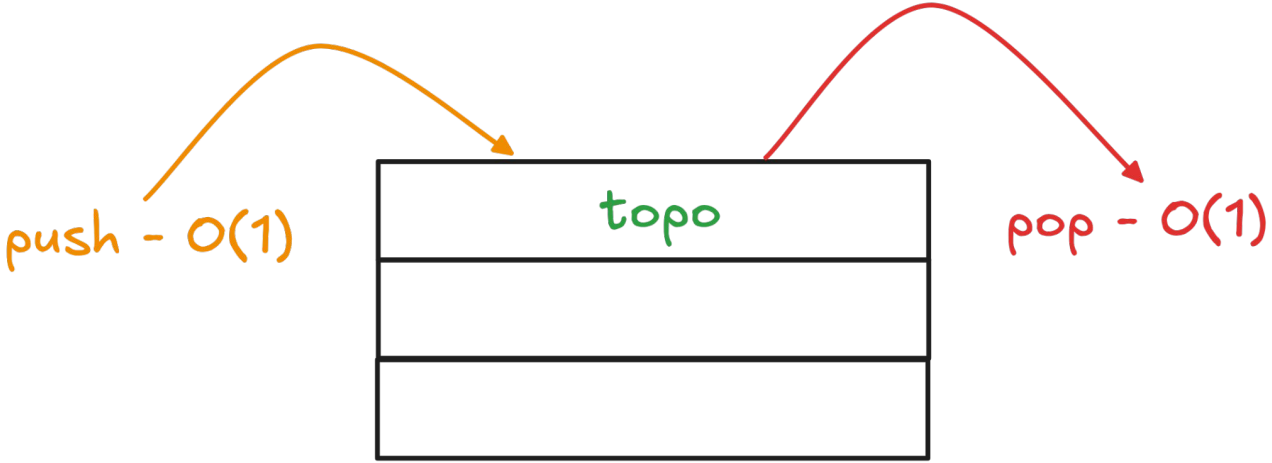
Pilhas

- Sempre trabalharemos com o elemento no topo da pilha
- Ou seja, pra inserir e para remover elementos na pilha a complexidade é O(1)



Pilhas

- Por ser uma estrutura de dados muito abstrata podemos usar listas encadeadas ou arrays para implementar nossas pilhas
- No caso da lista encadeada iriamos operar somente sobre a cabeça (head)
- Pilhas são estruturas de dados do tipo L.I.F.O (Last In, First Out)



Implementação da Pilha - Usando Listas Lineares

Uma forma comum de criação requer a definição de um tamanho fixo para a pilha, uma vez que só trabalharemos com os elementos no topo, pode ser que os elementos na base só ocupem espaço e nunca sejam utilizados.

Outra forma é ainda mais simples, não especificamos o tamanho e usamos como meio as listas da própria linguagem.

```
1 class Stack():
2     def __init__(self):
3         self.data = list()
4
5     def size(self):
6         return len(self.data)
7
8     def is_empty(self):
9         return not bool(self.size())
```

Implementação da Pilha - Usando Listas Lineares

Para adicionar um novo elemento utilizamos o método push e para remover o método pop.

```
def push(self, value):
    self.data.append(value)

def pop(self):
    if self.is_empty():
        return None

    value = self.data[-1]
    del self.data[-1]
    return value
```

carlos.melo@upe.br

Implementação da Pilha - Usando Listas Lineares

Também podemos criar um método para consultar o elemento no topo antes de fazermos alguma operação (*peek* ao invés de *get*)

```
1 def peek(self):
2     if self.is_empty():
3         return None
4     value = self.data[-1]
5     return value
```

Onde as pilhas são normalmente utilizadas?

- As linguagens de programação utilizam pilhas para controlar a ordem de chamada de funções (*CallStack*)

- Pilhas são tão comuns que o principal site para tirar dúvidas sobre programação se chama Stack Overflow, que é um erro comum quando lidamos com pilhas de tamanho limitado.

- O *browse* utiliza uma pilha para armazenar as paginas visitadas

- Pilhas são a base para algoritmos recursivos e dividir para conquistar (sobretudo algoritmos de ordenação)

Exercícios

Crie uma função para verificar se uma equação está com parênteses balanceados, ou seja, devo fechar cada parênteses aberto deve ter um fechando:

Exemplos:

- ((3 + 8)*(5/2))/(2+5) -> válido
- (3 + 8)*(5/2))/(2+5) -> **inválido**
- (3 + 8*(5/2))/(2+5) -> **inválido**

Crie 5 diferentes asserções para se certificar de seu funcionamento.

Exercícios

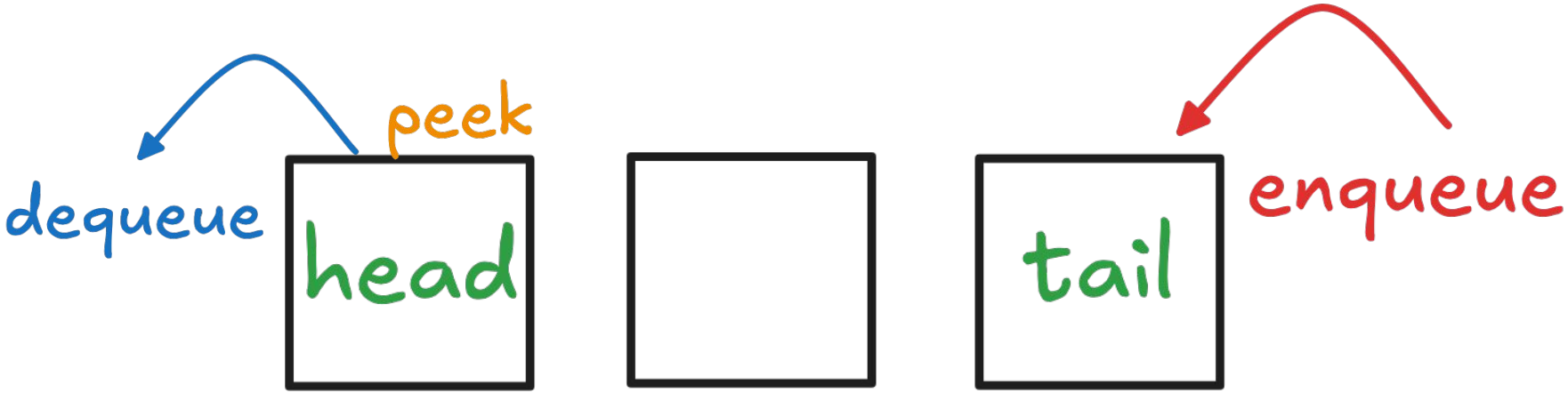
Implemente uma versão da pilha com limite de tamanho e caso esse seja ultrapassado lance uma exceção de StackOverflow

Filas

- Ninguém no mundo entende mais de fila que o brasileiro
- A ideia é sempre a mesma: quem está há mais tempo na fila, é quem será atendido primeiro
- Fila são estruturas de dados do tipo F.I.F.O (First In, First Out), ou seja, o oposto da pilha (L.I.F.O)
- As pessoas entram entram pela parte de trás da fila e saem pela frente

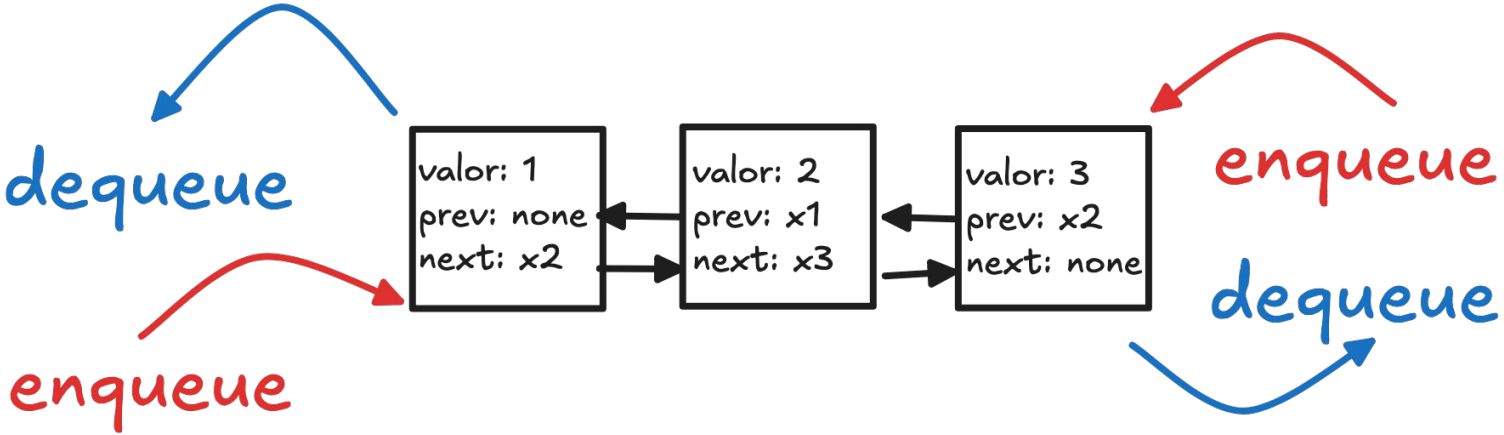
Filas

- O elemento mais antigo na fila é a cabeça, e o mais recente a cauda
- Quando adicionamos um elemento ao final chamos de enqueue (enfileirar)
- Quando removemos a cabeça fazemos um dequeue (desenfileirar)
- Para verificar o elemento na cabeça usamos o peek



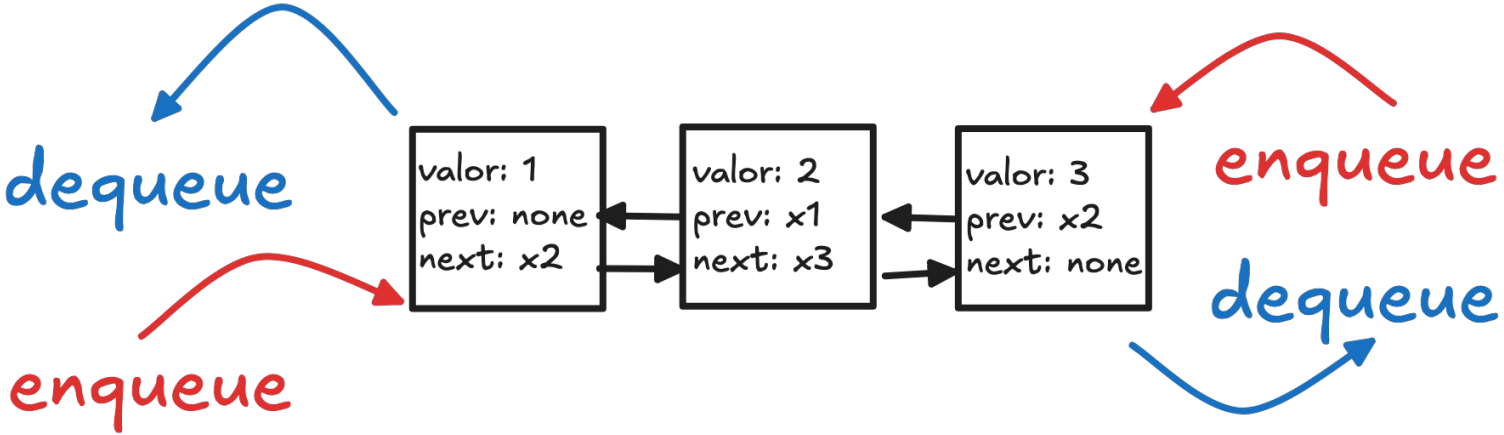
Filas

- Assim como as pilhas há muitas formas de se implementar uma fila
- Uma das mais comuns é por meio de listas duplamente encadeadas
  - Nesse caso chamamos a fila de deque (double ended queue)
- Também é possível implementá-las por meio de Listas Lineares (em sua versão mais simples)



Filas

- Uma deque é uma implementação que consegue representar tudo que fazemos numa pilha e numa fila



Implementando Filas - Com Listas Lineares

```
1 class Queue():
2     def __init__(self):
3         self.queue = list()
```

Implementando Filas - Com Listas Lineares

- Enfileirando Elementos

```
def enqueue(self, value):
    self.queue.append(value)
```

Implementando Filas - Com Listas Lineares

- Desenfileirando Elementos

```
1 def dequeue(self):
2     if len(self.queue) == 0:
3         return None
4     return self.queue.pop(0)
```

Exercícios

Implemente a fila com uma lista duplamente encadeada

Prof. Dr. Carlos Melo
https://calendly.com/prof-casm
carlos.melo@upe.br

UPE UNIVERSIDADE DE PERNAMBUCO CAMPUS GARANHUNS